Assignment 3:

Name: Leon Lobo

Admission No: U24AI043

1. Implement a Simple Reflex Agent for a vacuum cleaner that can navigate a three-room environment(A, B, and C) and maintain cleanliness. Identify the "rationality" of its actions. How will you define the performance cost? Define the complete Rule Table and store it either in a dictionary/vector/matrix based on the language you implement the vacuum cleaner. Show the simulation output with percept, action, and location. Do you need priorities in the rules?

Code:

A3_Q1.py > ...

```python
seed=int(input("Enter a random number for initial state:"))
choice=['Left','Right']
state=['Unclean','Clean']

def pseudo_random_number():
    global seed
    seed=(1103515245*seed+12345)%(2**31)
    return (seed>>16)&1

rule_table={
    ('A','Unclean'):'Cleanse',
    ('A','Clean'):'Right',
    ('B','Unclean'):'Cleanse',
    ('B','Clean'):choice[pseudo_random_number()],
    ('C','Unclean'):'Cleanse',
    ('C','Clean'):'Left'
}

environment={
    'A':state[pseudo_random_number()],
    'B':state[pseudo_random_number()],
    'C':state[pseudo_random_number()]
    }

print("\nInitial Environment State:",environment,end="\n\n")

room='A'
cost=0
steps=12

```

A3_Q1.py > ...

```python
    }

print("\nInitial Environment State:",environment,end="\n\n")

room='A'
cost=0
steps=12

for step in range(steps):
    status=environment[room]
    percept=(room,status)
    action=rule_table[percept]
    print(f"{step+1:>4}  {room:^8}  {status:^7}  {action}")
    if action=='Cleanse':
        environment[room]='Clean'
        cost+=20
    elif action=='Right':
        if room=='A':
            room='B'
        elif room=='B':
            room='C'
        cost+=5
    elif action=='Left':
        if room=='C':
            room='B'
        elif room=='B':
            room='A'
        cost+=5
print("\nFinal Environment State:",environment)
print("\nTotal Performance Cost:",cost)
```

Output:

```
Enter a random number for initial state:10

Initial Environment State: {'A': 'Unclean', 'B': 'Clean', 'C': 'Unclean'}

   1     A     Unclean  Cleanse
   2     A      Clean   Right
   3     B      Clean   Right
   4     C     Unclean  Cleanse
   5     C      Clean   Left
   6     B      Clean   Right
   7     C      Clean   Left
   8     B      Clean   Right
   9     C      Clean   Left
  10     B      Clean   Right
  11     C      Clean   Left
  12     B      Clean   Right

Final Environment State: {'A': 'Clean', 'B': 'Clean', 'C': 'Clean'}

Total Performance Cost: 90
```

Rationality: If the room is unclean, then the agent cleans it, if not it moves left/right depending on the current precept(location) and rule table.
Performance Cost: For cleaning a room, +20, and for moving left/right, +5.
Rule table:

A	Unclean	Cleanse
A	Clean	Right
B	Unclean	Cleanse
B	Clean	Left/Right(Random)
C	Unclean	Cleanse
C	Clean	Left

Priority: It is not required as only one signal is being sent to the agent at any one time and it does need to decide between multiple options at any given time.

2. Design a Simple Reflex Agent for an Indian Railways Level Crossing. The agent must ensure maximum safety for road traffic while minimizing delays for the train, using multi-source sensory input without historical tracking. The environment consists of a double-track railway line intersecting a busy state highway.
The percepts include the following:
● Track Sensors (Inbound/Outbound): Sensors placed 2km away that detect if a train is currently passing over them. It is either detected or not detected.

- Obstacle Sensors (Yellow Box): A grid of sensors on the road between the gates to detect “stuck” vehicles (e.g., an auto-rickshaw or buffalo).
- Manual Emergency Input: A physical lever for the station master. It is either Neutral or Active.

Actuators:

- Gate Arm: Lower/Raise
- Hooter/Siren: On/Off
- Signal to Train: Green (Safe to proceed)/Red (Emergency Stop)

Design the rules set. Show the simulation output with percept, action, and location. Do you need priorities in the rules?

Code:

A3_Q2.py > ...

```python
states=[
    {
        'sensor':'Inbound',
        'obstacle':False,
        'emergency':'Neutral'
    },
    {
        'sensor':'Inbound',
        'obstacle':True,
        'emergency':'Neutral'
    },
    {
        'sensor':'Outbound',
        'obstacle':True,
        'emergency':'Neutral'
    },
    {
        'sensor':'Outbound',
        'obstacle':False,
        'emergency':'Neutral'
    },
    {
        'sensor':'Inbound',
        'obstacle':False,
        'emergency':'Active'
    }
]
count=0
action={}
for state in states:
```

A3_Q2.py > ...

```python
            'sensor':'Inbound',
            'obstacle':False,
            'emergency':'Active'
        }
    ]
    count=0
    action={}
    for state in states:
        count+=1
        sensor=state['sensor']
        obstacle=state['obstacle']
        emergency=state['emergency']
        if emergency=='Active':
            action['gate']='Lower'
            action['siren']='On'
            action['train_signal']='Red'
        elif sensor=='Inbound':
            action['gate']='Lower'
            action['siren']='On'
            if obstacle:
                action['train_signal']='Red'
            else:
                action['train_signal']='Green'
        else:
            action['gate']='Raise'
            action['siren']='Off'
            action['train_signal']='Green'
        print(f"\nState {count}:-")
        print("Percept: ",state)
        print("Action : ",action)
```

Output:

```
State 1:-
Percept:  {'sensor': 'Inbound', 'obstacle': False, 'emergency': 'Neutral'}
Action :  {'gate': 'Lower', 'siren': 'On', 'train_signal': 'Green'}

State 2:-
Percept:  {'sensor': 'Inbound', 'obstacle': True, 'emergency': 'Neutral'}
Action :  {'gate': 'Lower', 'siren': 'On', 'train_signal': 'Red'}

State 3:-
Percept:  {'sensor': 'Outbound', 'obstacle': True, 'emergency': 'Neutral'}
Action :  {'gate': 'Raise', 'siren': 'Off', 'train_signal': 'Green'}

State 4:-
Percept:  {'sensor': 'Outbound', 'obstacle': False, 'emergency': 'Neutral'}
Action :  {'gate': 'Raise', 'siren': 'Off', 'train_signal': 'Green'}

State 5:-
Percept:  {'sensor': 'Inbound', 'obstacle': False, 'emergency': 'Active'}
Action :  {'gate': 'Lower', 'siren': 'On', 'train_signal': 'Red'}
```

There is a need for priority as different precepts can lead to a variety of states and hence conflicting information, for example: no obstacle and train outbound would imply the siren to be off and gate raised, however if there is an emergency, the gate must be lowered for the manual emergency lever.